INDIVIDUAL PROJECT REPORT

HACKAGON
A Web Platform for Hosting and Joining Online Coding Competitions

My modules: UI and Design System; Learning, Achievements and AI

Submitted in partial fulfilment of the requirements for the degree of

Bachelor of Technology in Computer Science and Engineering

Submitted by

Akshat Rathore (Member 5)
Roll No. 24EARCS014

Under the guidance of

Er. Rambabu Buri
Project Mentor, Department of CSE

Department of Computer Science and Engineering
Arya College of Engineering and IT, Jaipur

Session 2026-27

CERTIFICATE

This is to certify that the individual project report entitled “HackAgon: UI and Design System, Learning, Achievements and AI Modules”, submitted by Akshat Rathore (Roll No. 24EARCS014), a student of the third year of Bachelor of Technology in Computer Science and Engineering at Arya College of Engineering and IT, Jaipur, is a record of his own contribution to the team project HackAgon, carried out from 6 July 2026 to 6 October 2026 under the guidance and supervision of Er. Rambabu Buri, Department of CSE.

The work embodied in this report has not been submitted elsewhere for the award of any degree or diploma. The report is, in my opinion, worthy of consideration for the partial fulfilment of the requirements of the degree.

Er. Rambabu Buri — Project Mentor

Dr. Akhil Pandey — Head of Department

Date: _____

Place: Jaipur

DECLARATION

I, Akshat Rathore, Roll No. 24EARCS014, hereby declare that this individual project report entitled “HackAgon: UI and Design System, Learning, Achievements and AI Modules” is an original account of the work I did as Member 5 of the HackAgon project team, under the guidance of Er. Rambabu Buri.

I further declare that this report has not been copied from any other source. Wherever ideas, tools or text of others have been used, they are acknowledged in the references. The modules that were built by my teammates are mentioned only where my work depends on them.

Akshat Rathore

Date: _____

Place: Jaipur

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who helped me complete this work. I thank my project mentor, Er. Rambabu Buri, for direction, patient review and helpful suggestions. I am grateful to the Head of the Department, Dr. Akhil Pandey, and to the faculty of Arya College of Engineering and IT, Jaipur, for the learning environment and the laboratory facilities.

I thank my teammates Hiten Sharma, Harsh Sharma, Hemanth Singh Tanwar and Arjun Suman. Their modules gave my pages something real to show, and our regular discussions about interfaces, especially for the mistake book, the achievement counters and the money screens, made the integration smooth.

I also thank the open-source community and the authors of the free books listed in the HackAgon library. Finally, I thank my family and friends for their encouragement and for testing early versions of the screens and telling me honestly what was confusing.

Akshat Rathore

ABSTRACT

HackAgon is a web application for hosting and joining online coding competitions, built by a team of five students. This report describes my individual contribution as Member 5: the UI and Design System module and the Learning, Achievements and AI module.

The design system defines every colour, font, spacing step, radius, shadow and motion value once in theme.css, and a small set of stylesheets and scripts reuses them across fifteen pages. The learning features keep every failed question in a mistake book with the last code, the failing case and the right approach, allow a retry, offer a library of eleven free books, and track seven achievements from real activity. The AI coach, built on the Groq API, provides hints, explanations and coaching while keeping hidden test cases private, limiting each user to twelve calls a minute.

Testing with a local server passed fourteen API checks; live AI answers were not tested because no key was configured. A contrast audit found that one text colour, used for faint captions, reaches only 3.73 to 1, and a responsive check found one page that overflows at phone width. These findings and their fixes are reported openly.

Keywords: design system, CSS custom properties, accessibility, mistake book, achievements, AI coach, Groq, Java.

INDEX

CERTIFICATE.....2
DECLARATION.....3
ACKNOWLEDGEMENT.....4
ABSTRACT.....5
INDEX.....6
LIST OF FIGURES.....8
1. INTRODUCTION.....9
1.1 About the Project.....9
1.2 My Role and Modules.....9
1.3 Objectives of My Modules.....9
1.4 Scope and Limits of My Work.....10
1.5 Organisation of the Report.....10
2. BACKGROUND AND TOOLS.....11
2.1 Design Systems and CSS Custom Properties.....11
2.2 Accessibility Basics.....11
2.3 Learning from Mistakes.....11
2.4 Language Models as Coaches.....11
2.5 Tools Used.....11
3. REQUIREMENT ANALYSIS.....12
3.1 Functional Requirements.....12
3.2 Non-Functional Requirements.....12
3.3 Interfaces with Other Modules.....12
4. UI AND DESIGN SYSTEM.....14
4.1 Design Direction.....14
4.2 Colour Tokens.....14
4.3 Typography, Spacing and Motion.....14
4.4 Components.....15
4.5 Layout and Responsive Behaviour.....15
4.6 Accessibility Features Built In.....15
5. LEARNING AND ACHIEVEMENTS.....16
5.1 The Mistake Book.....16
5.2 Retry Flow.....16
5.3 The Library.....17
5.4 Achievements.....17
5.5 Tables Used.....18
6. AI COACH.....19
6.1 Design Goals.....19
6.2 Endpoints.....19
6.3 Prompt Design and Safety.....19
6.4 Limits and Failure Handling.....19
6.5 Front-End Integration.....20
6.6 What Is Not Done Yet.....20
7. FRONT-END INFRASTRUCTURE.....21
7.1 The API Wrapper (api.js).....21
7.2 The Data Layer (store.js).....21
7.3 The Shared Shell (app.js).....21
7.4 Why No Framework.....21
7.5 Pages I Built.....21
8. USER INTERFACE.....23
9. TESTING.....28
9.1 Approach and Environment.....28

9.2 API Tests..... 28
9.3 Contrast Audit..... 28
9.4 Responsive Check..... 29
9.5 Issues Found..... 29
10. WEEK-WISE WORK..... 30
11. CONCLUSION AND FUTURE SCOPE..... 31
11.1 Conclusion..... 31
11.2 What I Learnt..... 31
11.3 Limitations..... 31
11.4 Future Scope..... 31

LIST OF FIGURES

Figure 4.1: Layers of the design system....14
Figure 5.1: How a failure becomes learning....16
Figure 5.2: Recomputing achievements....18
Figure 6.1: Path of an AI coach request....20
Figure 8.1: Landing page....23
Figure 8.2: Log in and sign-up....23
Figure 8.3: The Arena....24
Figure 8.4: Create a room....24
Figure 8.5: Learning page, My mistakes....25
Figure 8.6: Learning page, Library....25
Figure 8.7: Profile with achievements....26
Figure 8.8: Tournaments....26
Figure 8.9: Phone layout (390 px wide): landing, Arena and Learning....27

1. INTRODUCTION

1.1 About the Project

HackAgon is a web application for hosting and joining online coding competitions. Any verified user can host a contest in solo, duo, squad or duel format, players join and solve questions that are judged automatically, and paid tournaments hold the prize pool and entry fees in escrow before paying winners by a published split after a flat 1% platform fee. A proctoring feature protects fairness, and a learning area helps players improve after every contest. The application is built with Java and Spring Boot, a MySQL-compatible database on TiDB Cloud, and fifteen HTML pages styled with plain CSS and JavaScript, and it is deployed from GitHub to Render.

The project was built by a team of five students between 6 July 2026 and 6 October 2026. The work was divided into ten modules, two for each member, so that everyone owned a complete part of the system from design to testing.

1.2 My Role and Modules

I am Member 5 of the team. My two modules are UI and Design System, and Learning, Achievements and AI. Together they decide how the whole product looks and feels, and how a player grows after each match.

Module	What it covers	Main files
UI and Design System	Colours, fonts, spacing, components, responsive layout, shared page shell and the data layer used by every page	css/theme.css, base.css, components.css, auth.css; js/api.js, store.js, app.js
Learning, Achievements and AI	Mistake book with retry, free book library, seven achievements, and the AI coach for hints and explanations	learning.html, profile.html; AiController.java; achievements in DomainService.java

In the week-wise plan I also built the pages that show money in the interface, namely the tournaments list and the tournament details page, and the landing page that welcomes new visitors.

1.3 Objectives of My Modules

- Give the product one consistent, human look defined in a single style-guide file, so that a change to one value changes every page.
- Make every page usable on a phone as well as on a laptop, with readable contrast and visible focus.
- Turn every failed attempt into a record the player can learn from, with a retry button and the right approach shown.
- Provide a free library of legally available books, and keep unverified books out until an administrator checks the licence.

- Reward progress with seven achievements whose progress comes only from real activity.
- Offer an AI coach that nudges a player towards the answer without leaking hidden test cases or handing over full solutions.

1.4 Scope and Limits of My Work

This report covers only my modules and the pages I built. The judge, rooms, money, proctoring and administration were built by the other members and are described in the team report. Some features in my area depend on outside services or on future work: the AI coach needs a Groq API key to answer, the library holds only the eleven seeded books until administrators approve more, and a few quality targets, such as contrast on faint text, are not fully met yet. These limits are stated openly in the chapters on testing and conclusion.

1.5 Organisation of the Report

Chapter 2 gives the background. Chapter 3 lists the requirements of my modules. Chapter 4 explains the design system, Chapter 5 the learning and achievement features, Chapter 6 the AI coach and Chapter 7 the front-end infrastructure. Chapter 8 shows the screens, Chapter 9 reports testing, Chapter 10 lists my week-wise work and Chapter 11 concludes.

2. BACKGROUND AND TOOLS

2.1 Design Systems and CSS Custom Properties

A design system is a shared set of decisions about colour, type, spacing and components that every screen of a product reuses. Without one, each page drifts: buttons get different shades, headings use different fonts, and a small change must be repeated everywhere. CSS custom properties make a design system practical on the web. A value such as a brand colour is declared once under the :root selector and referenced by name from every stylesheet, so a single edit updates the whole site.

HackAgon uses this approach without any framework. A single file, theme.css, holds the tokens, and the other stylesheets only use them. This keeps the project light, needs no build step, and lets the same Spring Boot server host the pages.

2.2 Accessibility Basics

The Web Content Accessibility Guidelines ask for a contrast ratio of at least 4.5 to 1 between normal text and its background, visible keyboard focus, and respect for a visitor's wish to reduce motion. Pages should also work at narrow widths and have meaningful labels for assistive tools. These ideas guided my work, and Chapter 9 measures how well the product meets them.

2.3 Learning from Mistakes

Practice platforms usually show a verdict and move on. Learners improve faster when a failed attempt is kept, explained and retried later. HackAgon therefore stores each failed question with the last code and the failing test, shows the correct approach beside it, and counts a later success as progress towards the Scholar achievement.

2.4 Language Models as Coaches

A large language model can explain a bug or suggest an approach in plain words. Two risks must be handled: the model might reveal answers that the player should find alone, and it might see hidden test data. HackAgon addresses both on the server. The prompt instructs the model to hint first and avoid full solutions, and the model receives only the failing input and a bug-class description, never the expected outputs. The model is reached through the Groq chat completions API.

2.5 Tools Used

Tool	Use in my modules
HTML, CSS, JavaScript	All pages, styles and the front-end data layer
Google Fonts	Fraunces, Karla, Kalam and JetBrains Mono
Groq API	Hints, explanations and coaching through the AI routes
Java, Spring Boot, MySQL	Back-end routes that my pages call
Git and GitHub	Version control and deployment trigger
Browser developer tools	Layout, responsive and focus checks

3. REQUIREMENT ANALYSIS

3.1 Functional Requirements

ID	Requirement
UI-1	All colours, fonts, spacing, radii, shadows and motion values shall be defined once in theme.css and reused by every page.
UI-2	Shared components (buttons, cards, badges, progress bars, tables, toasts, modals, empty states) shall be defined once in components.css and base.css.
UI-3	Every page shall load a shared shell with a top bar and, on small screens, a bottom navigation bar.
UI-4	The front end shall talk to the server only through one API wrapper that attaches the session token and turns errors into readable messages.
LRN-1	The Learning page shall list every failed question with difficulty, topic, last code, failing case and the right approach.
LRN-2	A player shall be able to retry a failed question from the page and see the verdict at once; a success shall mark the mistake solved.
LRN-3	The Library shall list approved books grouped by subject with the licence shown, and let a player suggest a book for administrator review.
ACH-1	Seven achievements shall show progress bars computed from real counts; a new account shall show zero everywhere.
AI-1	The system shall offer a hint, an explanation of a failed attempt, and general coaching through the Groq API.
AI-2	The AI routes shall never send hidden expected outputs to the model, shall limit each user to 12 calls per minute and shall require a signed-in user.

3.2 Non-Functional Requirements

- Consistency: no page may hard-code a brand colour or font that should come from the tokens.
- Accessibility: visible focus on every interactive control, reduced-motion support, and text contrast of at least 4.5 to 1 where possible.
- Responsiveness: usable at 390 px (phone), 768 px (tablet) and 1280 px (laptop) widths without sideways scrolling.
- Performance: no front-end framework and no build step; pages are static files served by the Spring Boot server.
- Safety: the browser never holds secrets; the Groq key stays on the server.

3.3 Interfaces with Other Modules

My module uses	Provided by	Purpose
/api/play/practice, /api/play/mistakes	Judge module (Member 2)	Retry a question and list mistakes
/api/play/achievements	Rooms and judge modules	Progress for the seven achievements

My module uses	Provided by	Purpose
/api/books, /api/admin/books	Content and admin modules	Library list, suggestions and approval
/api/auth/login, /api/auth/me	Accounts module (Member 1)	Session token and profile
/api/rooms, /api/money	Rooms and money modules	Data for tournament pages

4. UI AND DESIGN SYSTEM

4.1 Design Direction

The visual direction is called Warm and local: earthy, hand-made and human, as the header of theme.css puts it. The page background is a warm paper colour, cards have a thick ink edge and a hard offset shadow like a sticker, headings use a serif with character, and short handwritten labels add a personal touch. The aim was to avoid the generic glassy dashboard look and to make a student contest platform feel friendly.

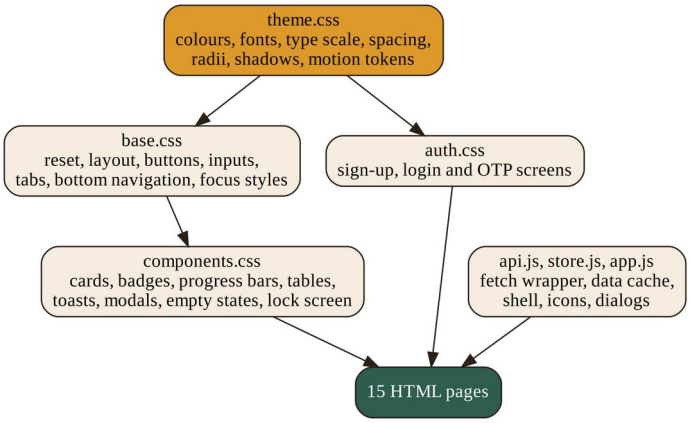


Figure 4.1: Layers of the design system

4.2 Colour Tokens

Token	Value	Role
--paper	#F6EDE0	Page background
--paper-2	#FBF6EC	Card surface
--ink	#2A211B	Body text
--ink-soft	#5C4C3F	Muted text
--terracotta	#B4523A	Primary accent and buttons
--turmeric	#DD9A2B	Secondary accent and highlights
--forest	#2F5D50	Success and secondary buttons
--stamp	#9E3324	Errors, live badge, lock screen
--sky	#4C7C8C	Information

Semantic tokens such as --bg, --surface, --text, --primary, --accent, --success and --danger point to these core values, so components talk about meaning rather than colour names.

4.3 Typography, Spacing and Motion

- Fonts: Fraunces for display headings, Karla for body text, Kalam for handwritten accents and JetBrains Mono for code.
- Type scale: seven steps from 0.78 rem to a fluid headline that grows with the screen width.
- Spacing: nine steps from 0.25 rem to 6 rem, so layouts keep one rhythm.

- Radii: 6, 12, 18 and 26 pixels, with a full round tag shape for chips.
- Shadows: two soft lifts and two hard sticker offsets.
- Motion: three durations (140, 240 and 520 ms) with one easing curve; all become near zero when the visitor prefers reduced motion.

4.4 Components

Component	Use
Card, sticker card, note card, well	Containers for rooms, mistakes, forms and notes
Tile	Room and tournament cards that lift on hover
Badges	Live, open, full, paid, free, proctored, ended and draft states
Difficulty pill	Easy, medium and hard with a coloured dot
Avatar	Initials in five colours
Progress bar	Achievement and quota progress with a hatched fill
Table	Scoreboards and admin lists, with the player's own row marked
Toast	Success, error, information and development messages
Modal and side sheet	Retry dialog, confirmations, details
Empty state	Illustrated message with a call to action
Countdown and lock screen	Match start animation and the proctoring lock
Buttons, inputs, tabs, switches, segmented control	Forms and navigation

4.5 Layout and Responsive Behaviour

Content is centred in a column no wider than 1180 pixels under a sixty-pixel top bar. Two breakpoints, at 900 and 620 pixels, reflow grids and enlarge touch targets. On phones a bottom navigation bar offers Tournaments, Arena, Duel, Learning and Profile, with a prominent Host button. Toasts move to full width above the bar on small screens.

4.6 Accessibility Features Built In

- A skip link on twelve of the fifteen pages and a screen-reader-only text class.
- A visible turmeric focus outline on checkboxes, switches, segmented controls and fields.
- Reduced-motion media query that shortens every animation and transition.
- ARIA roles and labels on tabs and dialogs; 25 aria-attributes across the pages.
- Icons marked aria-hidden so that screen readers skip decoration.

5. LEARNING AND ACHIEVEMENTS

5.1 The Mistake Book

The Learning page opens on My mistakes. Each failed question appears as a card with its difficulty, topic, how long ago it happened, the last code the player wrote, where it broke (for example, Test case 1 failed) and the right approach. A green label, solved since, shows when the player has fixed it later. Two buttons sit on every card: Ask the coach and Retry.

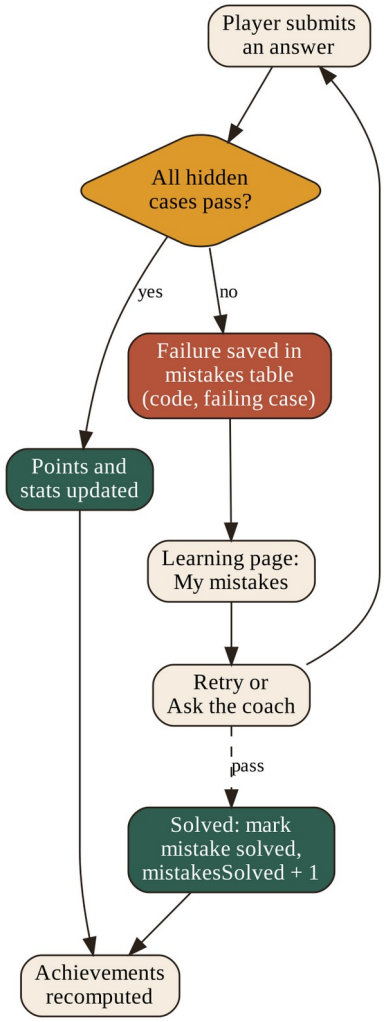


Figure 5.1: How a failure becomes learning

The server saves the failure automatically when a judged attempt fails, in the mistakes table with the question, the last code and the failing case. The page reads the list from /api/play/mistakes. The hidden test inputs never reach the browser; the page only shows the number of the failing case.

5.2 Retry Flow

Retry opens a dialog with an editor pre-filled with the last code, or a starter template in the chosen language. The player runs the code, and the dialog shows the verdict and the number of cases passed. When all cases pass, the server marks the mistake solved, increases the

mistakesSolved statistic and recomputes the achievements. The page then thanks the player with a short message and removes the card from the unsolved list.

5.3 The Library

The Library tab groups books by subject. The seeded shelf has eleven books in seven subjects: Algorithms, C, Competitive programming, Data structures, Java, JavaScript and Python. Each card shows the title, author, licence tag and an Open button. Examples are Automate the Boring Stuff with Python, Eloquent JavaScript, Think Java, Open Data Structures, Algorithms by Jeff Erickson and The Competitive Programmer's Handbook.

A form lets any player suggest a book. The suggestion goes to an administrator who checks the licence before it reaches the shelf, and scans of paid books are rejected. This protects the platform from hosting material that is not freely and legally available.

5.4 Achievements

Achievement	Statistic	Target
First Blood	roomsWon	1
Room Master	roomsWon	20
Tournament Conqueror	tournamentsWon	1
Duelist	duelsWon	10
Perfectionist	perfectRooms	1
Clean Player	cleanProctored	10
Scholar	mistakesSolved	25

Each achievement has a statistic and a target stored in the achievements table, so thresholds can change without changing code. Progress is the smaller of the statistic and the target. Progress for each user is stored in ach_state, together with the time the achievement was unlocked.

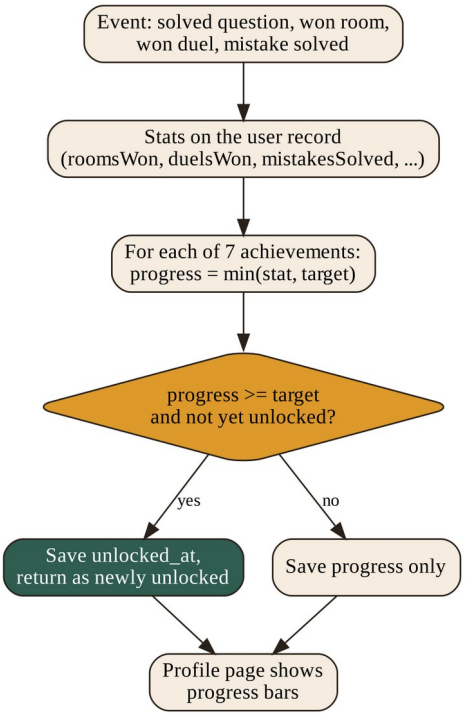


Figure 5.2: Recomputing achievements

5.5 Tables Used

Table	Columns used by my modules
mistakes	user, question, last code, failing case, solved flag, time
books	id, title, author, subject, licence, url, status (approved, pending, rejected), submitted_by
achievements	id, name, description, metric, target
ach_state	user_id, achievement_id, progress, unlocked_at

6. AI COACH

6.1 Design Goals

The AI coach should feel like a patient senior student. It gives one useful nudge, stays short, and never spoils the exercise. It must work only for signed-in users, must not be abused, and must keep the hidden tests private. The coach is implemented in AiController.java with a small Groq client inside it.

6.2 Endpoints

Endpoint	Purpose	Input
POST /api/ai/hint	A nudge towards the right data structure or recurrence; after two failures, an outline and a short skeleton	questionId
POST /api/ai/explain	Explains why a submission failed, after re-judging it on the server	questionId, lang, code
POST /api/ai/coach	General study help for a topic or a mistake in the book	topic, question, mistakeId
GET /api/ai/status	Tells the page whether the coach is configured	none

6.3 Prompt Design and Safety

Every call starts with a system message that sets the persona and the rules. In short, the coach must be concrete, encouraging and brief; must never paste a full solution unless the student asks after failing twice; must never reveal expected outputs and describe only the shape of the failing input and the class of bug; must answer in at most three short markdown sections; and must keep code snippets under fifteen lines.

For an explanation, the server first re-runs the judge on the submitted code, so the explanation matches reality and not the client's claim. It then sends the language, the code, the number of passed cases and a description of up to two failing cases. Code longer than 20,000 characters is refused. For coaching, a mistake id adds context such as the title, topics and difficulty, without hidden data.

6.4 Limits and Failure Handling

- Rate limit: a user may make 12 AI calls in any 60 seconds; the next call receives HTTP 429 with a friendly message.
- Timeout: the call to Groq is aborted after 30 seconds.
- Token limits: 1,400 to 1,600 tokens per answer, with headroom because reasoning models spend some tokens thinking.
- Missing key: the server returns HTTP 503 with a clear message instead of failing silently.
- Groq errors become HTTP 502, so the page can say that the coach is unavailable.

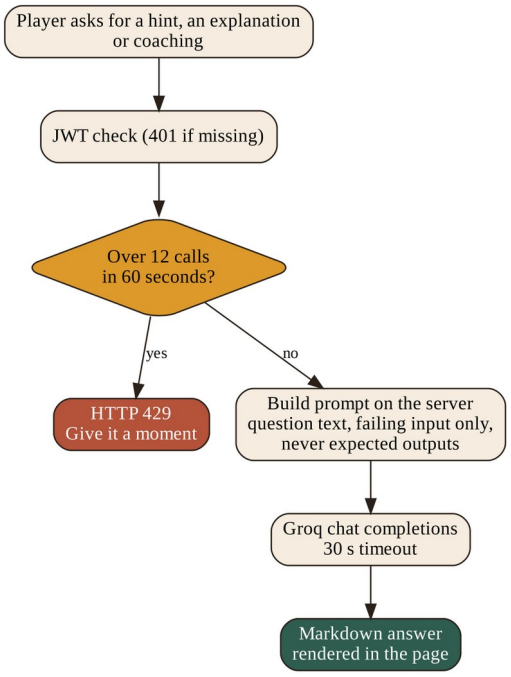


Figure 6.1: Path of an AI coach request

6.5 Front-End Integration

The match page offers Hint and Explain buttons after a failed attempt. The Learning page offers Ask the coach on each mistake card. Answers arrive as markdown, which a small converter in app.js turns into safe HTML with paragraphs, lists, bold text and code, escaping everything first so that model output cannot inject markup. The model name returned by the server is shown beside the answer.

6.6 What Is Not Done Yet

The original plan asked for daily limits per user. The build limits calls per minute only. A daily quota would protect the free Groq allowance better and is listed in the future scope. The coach also could not be tested with live answers during my testing because no Groq key was configured on the test server; all other behaviour of the routes was tested.

7. FRONT-END INFRASTRUCTURE

7.1 The API Wrapper (api.js)

api.js is about sixty lines. It stores the session token in the browser under the key hackagon.token, adds it as a bearer token to every request, and wraps fetch so that every failure becomes an ApiError with a status, a message and, for forms, a map of field errors. A 401 response clears the stored token so the page can send the player to log in. If the server cannot be reached, the message tells the developer to start the server. Storage access is wrapped in try and catch so that private browsing does not break the page.

```
const token = opts.anon ? null : getToken();
if (token) headers.Authorization = "Bearer " + token;
...
if (res.status === 401 && !opts.anon) setToken(null);
```

7.2 The Data Layer (store.js)

store.js is about five hundred lines. It keeps a read cache so pages can render at once, and every change is a request to the server followed by a cache refresh. A boot call loads content such as questions, books, achievements, topics, languages and prize presets from one bootstrap endpoint. The functions are grouped by area: accounts and OTP, rooms, play and mistakes, proctoring, money, library, administration and AI. Money is handled as integer paise end to end, and a helper formats it as rupees for display.

7.3 The Shared Shell (app.js)

app.js, about 370 lines, builds what every page shares. It draws the top bar and the bottom navigation, guards pages that need a signed-in user, shows toasts and dialogs, and provides small helpers such as an element builder, an HTML escaper, an avatar maker, status badges, a difficulty pill and a price tag. It contains a set of 36 hand-drawn inline SVG icons, so no icon library is loaded. A page function wraps the common start-up steps, and an act helper disables a button while a request runs and shows errors on the right fields.

7.4 Why No Framework

The team chose plain HTML, CSS and JavaScript. This keeps deployment simple, because the Spring Boot server serves the files directly, avoids a build step on free hosting, and lets any teammate read every page without learning a framework. The cost is some repeated markup, which the shared shell and the component stylesheet reduce.

7.5 Pages I Built

Page	Purpose
index.html	Landing page
login.html, signup.html, verify-otp.html	Account screens with hand-written copy
arena.html, create-room.html, create-tournament.html	Browse rooms and create rooms and tournaments
match.html	Editor, question list, timer, scoreboard and countdown

Page	Purpose
learning.html	Mistake book and library
profile.html	Details, statistics and achievements
tournaments.html, tournament-details.html	List and rules of paid tournaments

8. USER INTERFACE

The screens below were captured from a local test run with sample data. They show the design system at work on the pages I built.

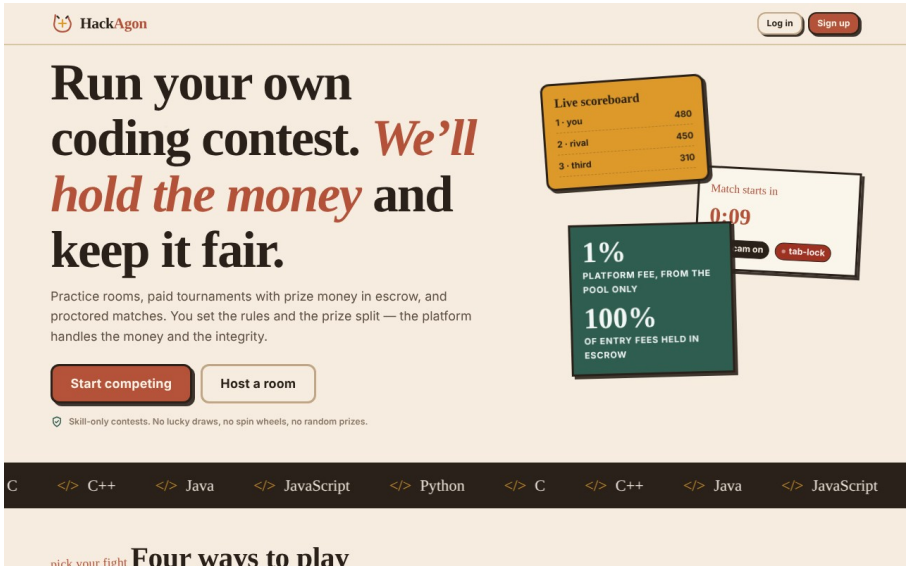


Figure 8.1: Landing page

The landing page states the idea in one screen and uses the sticker-style scoreboard card to hint at live matches.

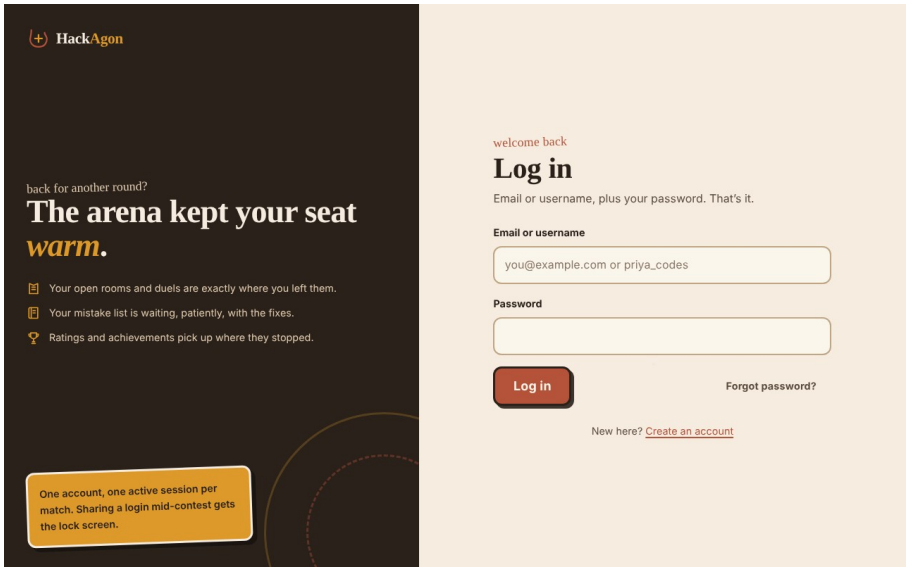


Figure 8.2: Log in and sign-up

The account screens use a dark panel with friendly copy and the OTP flow described in the team report.

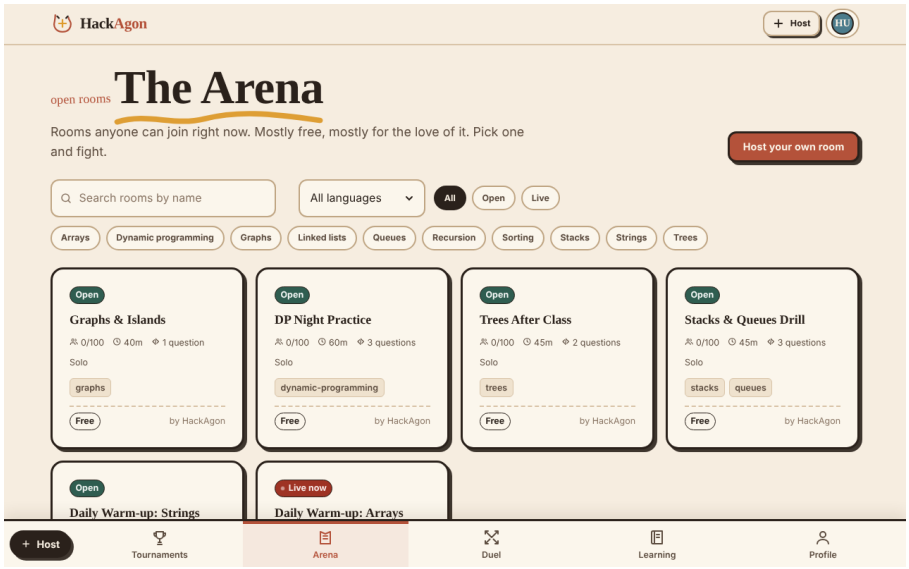


Figure 8.3: The Arena

Room cards with status badges, a search box, a language filter and topic chips.

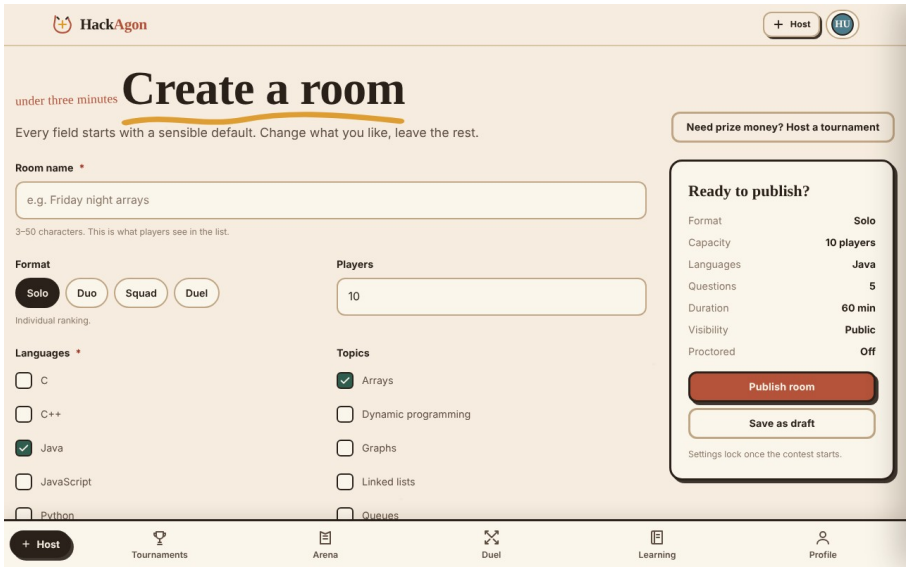


Figure 8.4: Create a room

A form with defaults and a summary card on the right that updates as the host changes settings.



Figure 8.5: Learning page, My mistakes

A mistake card with the last code, where it broke, the right approach, and Ask the coach and Retry buttons.

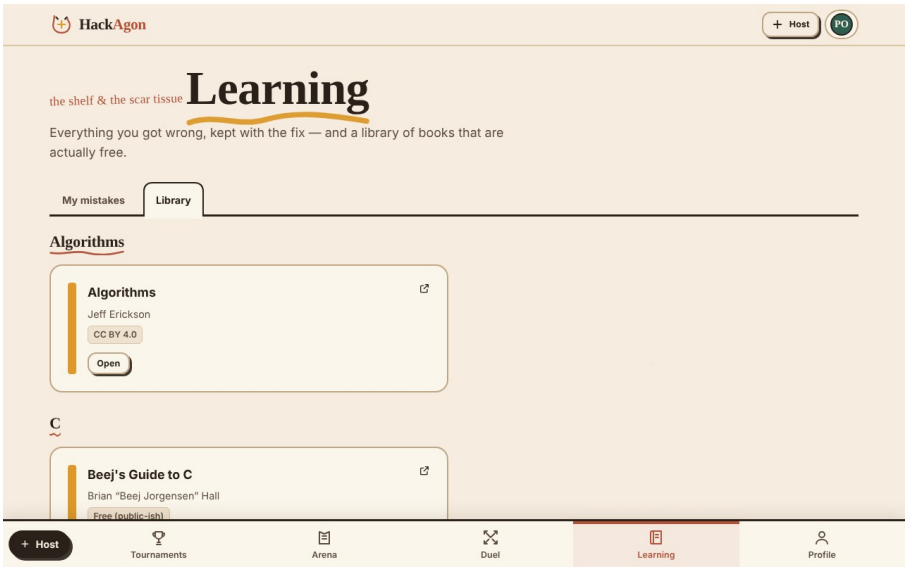


Figure 8.6: Learning page, Library

Books grouped by subject, each with a licence tag and an Open button.

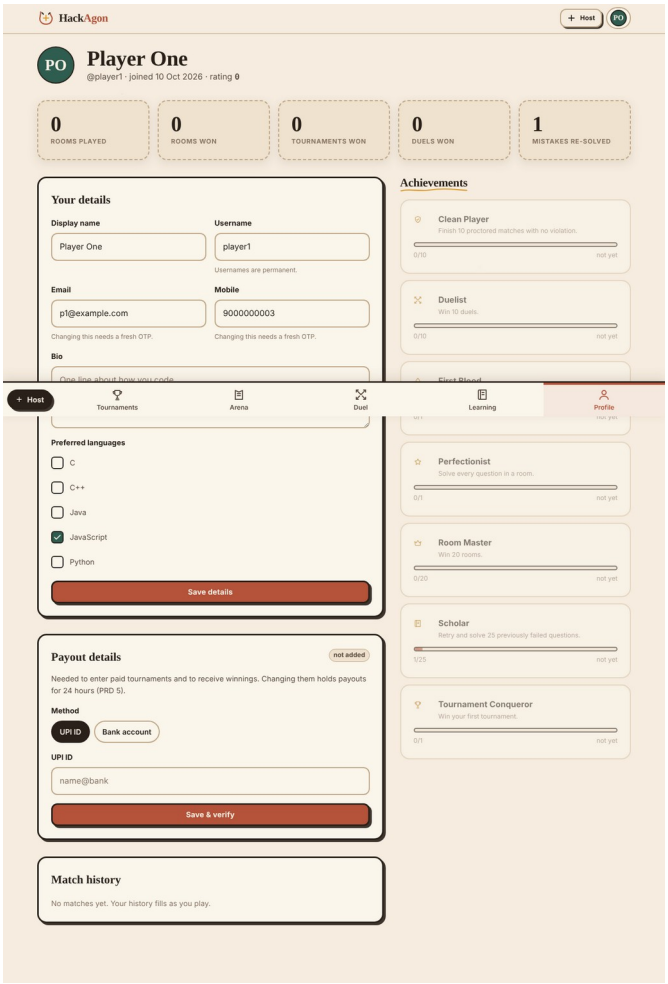


Figure 8.7: Profile with achievements

Counters that start at zero, editable details and achievement progress bars.

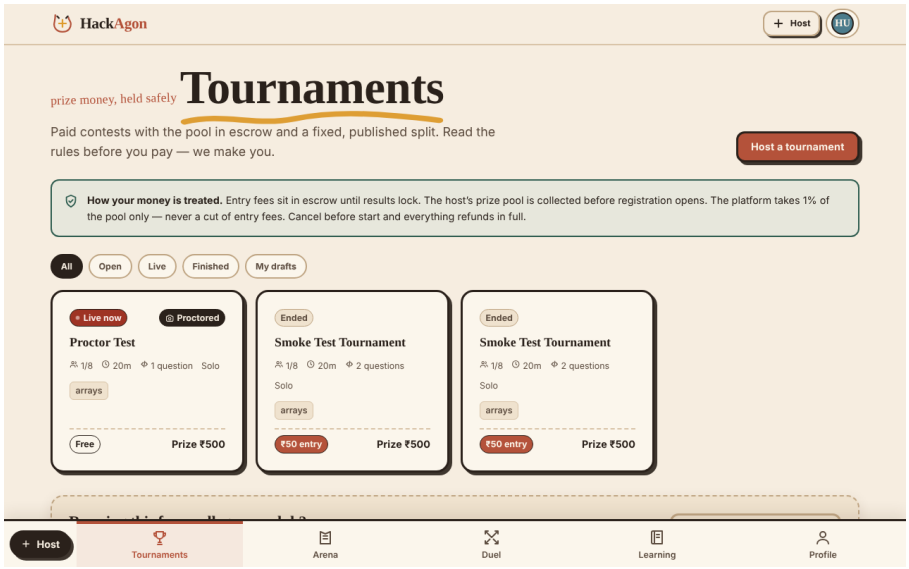


Figure 8.8: Tournaments

Paid contests with a plain-language notice about escrow and the 1% platform fee.

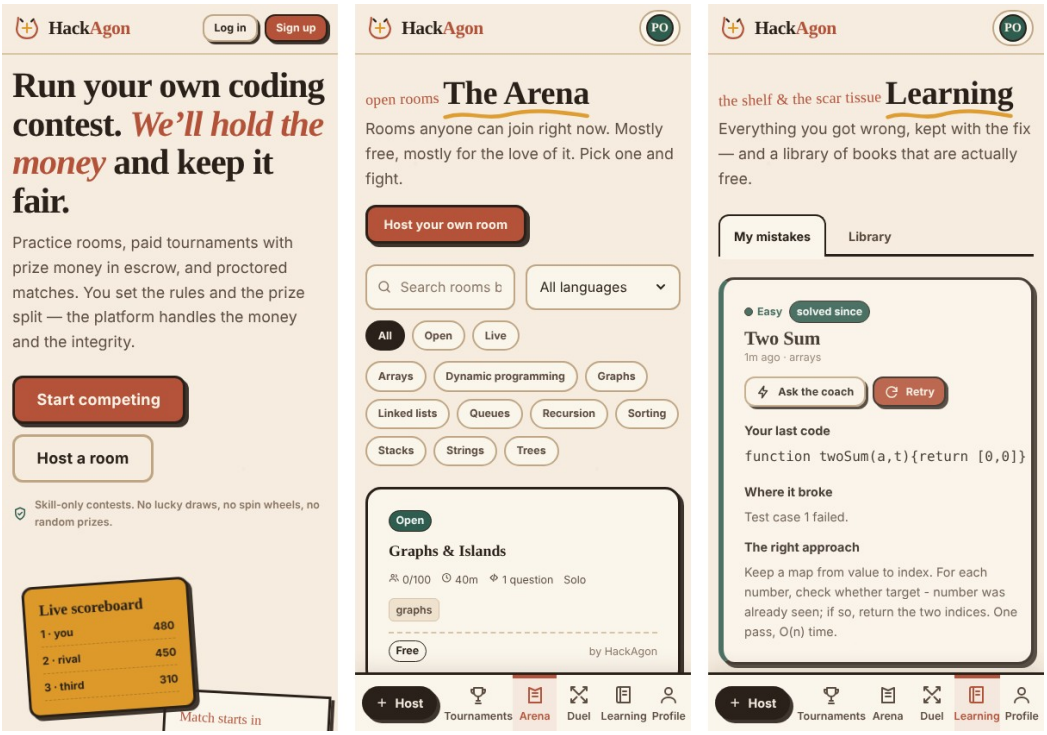


Figure 8.9: Phone layout (390 px wide): landing, Arena and Learning

On phones the top menu gives way to the bottom navigation bar, and cards stack into one column.

9. TESTING

9.1 Approach and Environment

The routes behind my modules were tested with a script that signs in as a fresh player and calls the API in sequence. The test server ran locally against MariaDB 10.11 on the earlier Java build of the server (the Spring Boot build has not yet been run through this script) and no Groq key. In addition I rendered ten pages at three screen widths in a headless Chromium browser to look for sideways overflow, and I calculated the contrast ratio of the main colour pairs from the token values.

9.2 API Tests

No.	Test case	Result
1	Achievements catalogue returns seven achievements	Pass
2	A new account shows 0 progress on every achievement	Pass
3	Library lists eleven approved books	Pass
4	Mistake book is empty for a new account	Pass
5	A wrong practice answer fails (0 of 4 cases)	Pass
6	The failure is saved in the mistake book	Pass
7	A correct retry passes 4 of 4 cases and reports mistakeSolved	Pass
8	The mistake is marked solved after the retry	Pass
9	Scholar progress rises to 1 of 25 (4%)	Pass
10	AI status endpoint reports whether the coach is configured	Pass
11	AI hint without a key returns HTTP 503 with a clear message	Pass
12	After 12 calls in a minute the next AI call returns HTTP 429	Pass
13	AI explain refuses empty code with HTTP 400	Pass
14	AI routes refuse a signed-out user with HTTP 401	Pass
15	AI hint and explanation with live model answers	Not run (no key)

9.3 Contrast Audit

Pair	Ratio	Meets 4.5 to 1
Body text on page background	13.60	Yes
Body text on card surface	14.64	Yes
Muted text on page background	7.07	Yes
Button text on forest green	6.81	Yes
Text on stamp red (live badge)	6.23	Yes
Dark text on turmeric	6.56	Yes
Button text on terracotta	4.53	Yes, barely
Faint text (captions, placeholders) on page background	3.73	No

9.4 Responsive Check

Ten pages (home, log in, sign-up, Arena, create room, create tournament, tournaments, learning, profile and host dashboard) were loaded at 390, 768 and 1280 pixels. At 768 and 1280 pixels no page scrolled sideways. At 390 pixels nine pages fitted, and the Learning page overflowed by about 105 pixels.

9.5 Issues Found

- Learning page at phone width: the block that shows the player's last code does not wrap or scroll when a line is long, so the page becomes wider than the screen. The fix is to let that block scroll sideways inside its card.
- Faint text colour reaches only 3.73 to 1. It is used for captions and placeholders; it should be darkened to about #6F5D4E or darker.
- Daily AI limits were planned but only the per-minute limit exists.
- The AI coach needs a live test with a Groq key before a demonstration.

10. WEEK-WISE WORK

The project ran from 6 July 2026 to 6 October 2026. The first twelve weeks were organised as weekly work packages, and the last days were used for the final review and submission. The table lists my planned work in each week and what exists in the final build.

Week	Dates	Planned work (Member 5)	Result in the build
1	6 to 12 Jul	Design direction: references, three previews of the Arena page, style guide and first tokens	theme.css style guide with all tokens
2	13 to 19 Jul	Components, base layout, navigation, login, sign-up and verify pages	components.css, base.css, auth.css and the three account pages
3	20 to 26 Jul	Create-room page with question panel, Arena cards, empty states	create-room.html, arena.html, empty-state component
4	27 Jul to 2 Aug	Match page: editor, question list, timer, scoreboard, countdown	match.html with the countdown animation
5	3 to 9 Aug	My mistakes list with last attempt, failing case and retry	Mistake book on learning.html with retry dialog
6	10 to 16 Aug	Consent, camera check and lock screens; free book library by subject	Lock screen and camera styles; Library tab with licences
7	17 to 23 Aug	Achievements with progress bars; AI hints through the AI route	Seven achievements with progress bars; AiController.java routes
8	24 to 30 Aug	Tournaments list, rules page, checkout, payout status, winner screen	Tournaments list and details pages; payment is simulated
9	31 Aug to 6 Sep	AI hints with limits; design consistency pass	Hint and explain in match.html; 12 calls per minute limit
10	7 to 13 Sep	Accessibility and copy: contrast, focus states, phone width	Focus styles, reduced motion, skip links; one contrast pair below target
11	14 to 20 Sep	Landing page with real content, onboarding hints, screenshots, demo script	index.html landing page; screenshots in this report
12	21 to 27 Sep	Fix last visual issues; demo and presentation; collect feedback	Final polish; report, SRS and README prepared with the team

Final review and submission took place from 28 September to 6 October 2026.

11. CONCLUSION AND FUTURE SCOPE

11.1 Conclusion

My modules gave HackAgon its face and its habit of teaching. The design system keeps fifteen pages consistent through one token file, one component stylesheet and one shared shell. The learning features turn each failure into a retry, a pointer to the right approach and, in time, an achievement. The AI coach adds help on demand while keeping the hidden tests private.

Testing showed that the routes behind my modules behave as designed, and it also showed where work remains: one phone-width overflow, one low-contrast colour, no daily AI quota and no live AI test yet.

11.2 What I Learnt

- A single source of truth for style makes large changes cheap and keeps a team's pages in step.
- Accessibility has to be measured, not assumed; a simple calculation revealed a colour that looked fine but missed the target.
- Working with an outside AI service needs rate limits, timeouts and clear failure messages from the first day.
- Clear module ownership and agreed interfaces with teammates prevented conflicts, especially around the mistake hook in the judge and the achievement counters.

11.3 Limitations

- The AI coach needs a Groq key and was not tested with live answers.
- There is no daily AI quota.
- Faint text does not meet the 4.5 to 1 contrast target, and the Learning page overflows at phone width in one case.
- Library growth depends on administrators approving suggestions.
- The money screens use a simulated payment flow.

11.4 Future Scope

- Fix the overflow and darken the faint text colour.
- Add a daily AI quota per user and show remaining calls on the page.
- Add an unlock animation and a share card for achievements.
- Add a dark theme using the same tokens.
- Add automated visual and accessibility checks to the deployment pipeline.

REFERENCES

[1] Oracle. Java SE 17 Documentation. https://docs.oracle.com/en/java/javase/17/

[2] VMware. Spring Boot 3.2 Reference Documentation. https://docs.spring.io/spring-boot/docs/3.2.0/reference/html/

[3] MySQL 8.0 Reference Manual. https://dev.mysql.com/doc

[4] World Wide Web Consortium. Web Content Accessibility Guidelines (WCAG) 2.1, success criterion 1.4.3 Contrast (Minimum). https://www.w3.org/TR/WCAG21

[5] Mozilla Developer Network. Using CSS custom properties; prefers-reduced-motion. https://developer.mozilla.org

[6] Groq. GroqCloud API documentation. https://console.groq.com/docs

[7] Google Fonts. Fraunces, Karla, Kalam and JetBrains Mono. https://fonts.google.com

[8] HackAgon repository. https://github.com/ASR-C/HACK-AGON